

Assignment-10.4

2303A51242

Batch-05

Task 1: AI-Assisted Syntax and Code Quality Review

Wrong code:

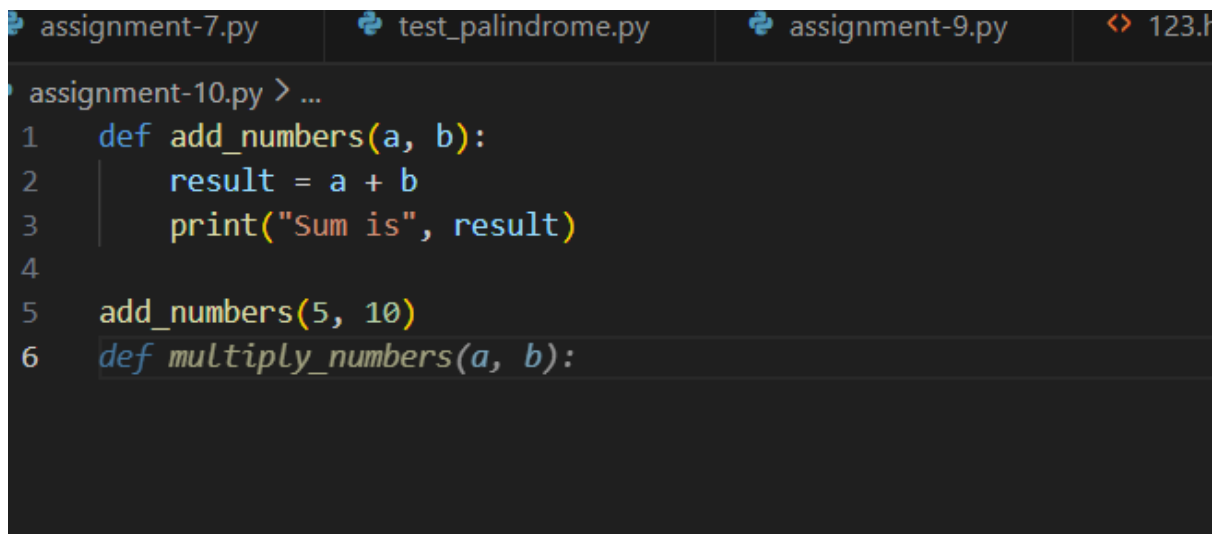
```
assignment-10.py > ...
1  def addNumbers(a,b)
2  result=a+b
3  print("Sum is",result)
4  addnumbers(5,10)
5  |
```

Output:

```
PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-10.py"
File "c:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING\assignment-10.py", line 1
    def addNumbers(a,b)
                        ^
SyntaxError: expected ':'
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

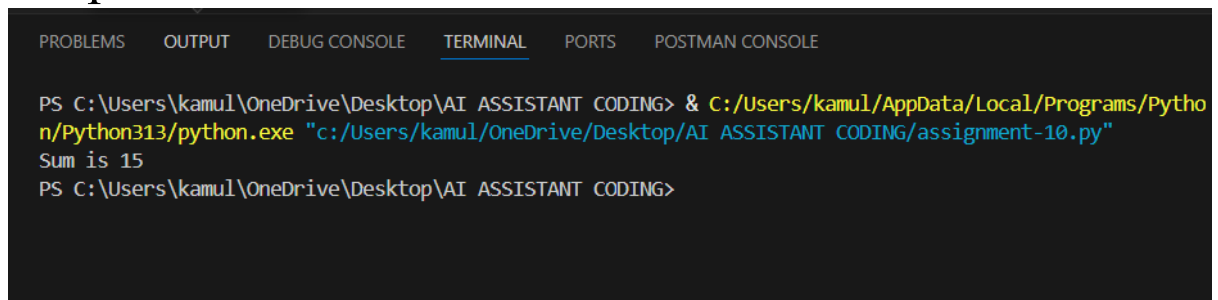
- Missing colon (:) in function definition
- Incorrect indentation
- Function name mismatch (addNumbers vs addnumbers)

Corrected code:



```
assignment-7.py  test_palindrome.py  assignment-9.py  123.f
assignment-10.py > ...
1  def add_numbers(a, b):
2      result = a + b
3      print("Sum is", result)
4
5  add_numbers(5, 10)
6  def multiply_numbers(a, b):
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE

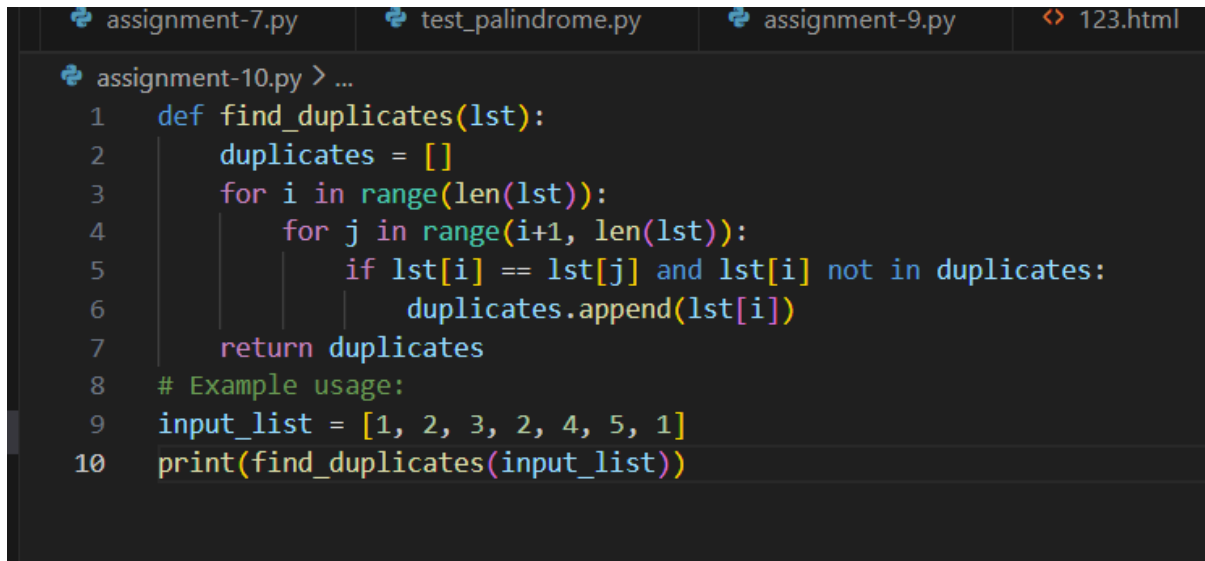
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-10.py"
Sum is 15
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation:

- Added missing colon to fix syntax error
- Corrected indentation to avoid runtime errors
- Standardized function naming using snake_case
- Ensured function call matches definition
- Improved readability

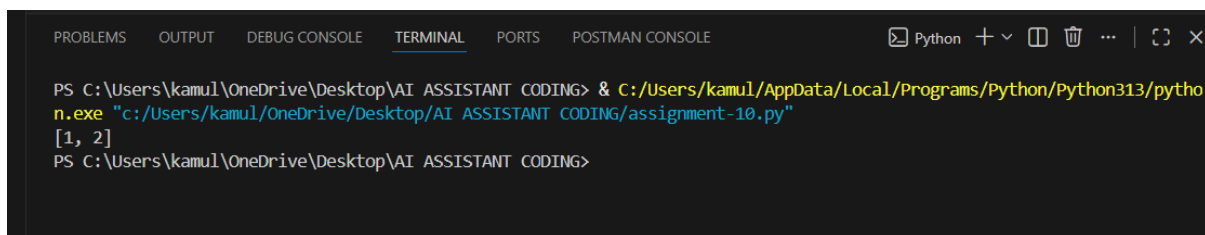
Task 2: Performance-Oriented Code Review

Inefficient code:



```
assignment-7.py  test_palindrome.py  assignment-9.py  123.html
assignment-10.py > ...
1  def find_duplicates(lst):
2      duplicates = []
3      for i in range(len(lst)):
4          for j in range(i+1, len(lst)):
5              if lst[i] == lst[j] and lst[i] not in duplicates:
6                  duplicates.append(lst[i])
7      return duplicates
8  # Example usage:
9  input_list = [1, 2, 3, 2, 4, 5, 1]
10 print(find_duplicates(input_list))
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE
Python + - [ ] [ ] ... | [ ] [ ] X
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & c:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-10.py"
[1, 2]
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

- Uses nested loops → $O(n^2)$ time complexity
- Poor performance for large datasets

Optimized code:

```
assignment-7.py test_palindrome.py assignment-9.py 123.html assignment-10.py
```

```
assignment-10.py > ...
1  def find_duplicates(lst):
2      seen = set()
3      duplicates = set()
4
5      for item in lst:
6          if item in seen:
7              duplicates.add(item)
8          else:
9              seen.add(item)
10
11     return list(duplicates)
12 # Example usage:
13 if __name__ == "__main__":
14     sample_list = [1, 2, 3, 4, 2, 5, 1, 6, 3]
15     print("Duplicates in the list:", find_duplicates(sample_list))
16
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
Python + - [ ] [ ] ... [ ] [ ]
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-10.py"
Duplicates in the list: [1, 2, 3]
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation:

- Original approach was inefficient due to nested loops
- Sets provide **O(1)** lookup time
- Optimized solution reduces complexity to **O(n)**
- Maintains correctness with better scalability

Task 3: Readability and Maintainability Refactoring

Poorly Structured Code:

```
assignment-10.py > ...
1  def f(x):
2      if x>0:
3          return x*x
4      else:
5          return x+x
6  print(f(2))
7  print(f(-2))
```

Output:

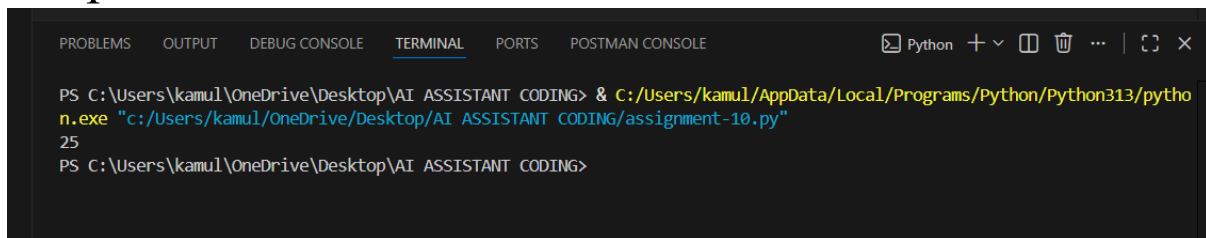
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE
Python + v [Icons] ... | [Icons] X

PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-10.py"
4
-4
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Refactored Code:

```
assignment-10.py > ...
1  def calculate_value(number):
2      """
3      Calculates square if number is positive,
4      otherwise returns double the value.
5      """
6      if number > 0:
7          return number ** 2
8      return number * 2
9  # Test the function with different inputs
10 print(calculate_value(5)) # Output: 25
```

Output:



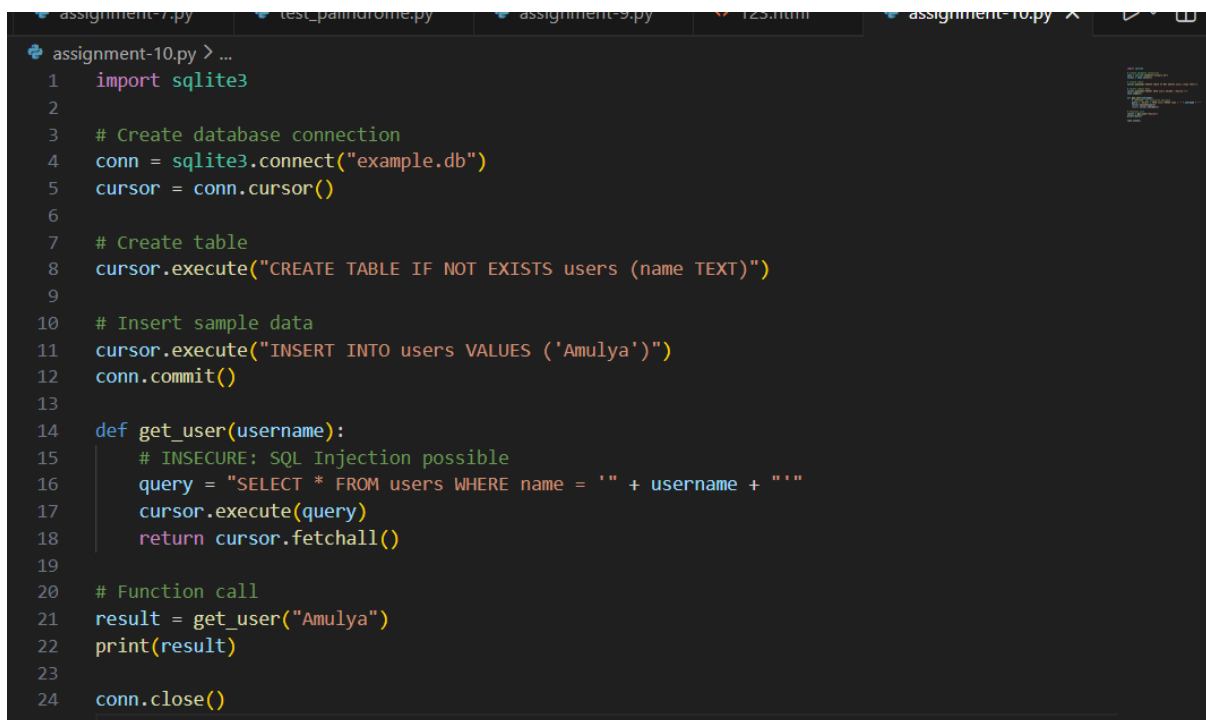
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
Python + - [ ] [X] ... | [ ] [X]
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-10.py"
25
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation:

- Descriptive function and variable names
- Proper indentation and formatting
- Added meaningful docstring
- Simplified logic flow
- Improved clarity and long-term maintainability
- Easier for new developers to understand

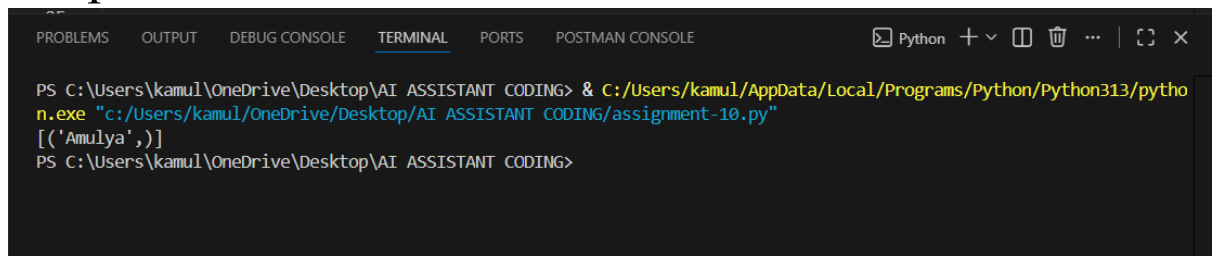
Task 4: Secure Coding and Reliability Review

Insecure Code:



```
assignment-10.py > ...
1  import sqlite3
2
3  # Create database connection
4  conn = sqlite3.connect("example.db")
5  cursor = conn.cursor()
6
7  # Create table
8  cursor.execute("CREATE TABLE IF NOT EXISTS users (name TEXT)")
9
10 # Insert sample data
11 cursor.execute("INSERT INTO users VALUES ('Amulya')")
12 conn.commit()
13
14 def get_user(username):
15     # INSECURE: SQL Injection possible
16     query = "SELECT * FROM users WHERE name = '" + username + "'"
17     cursor.execute(query)
18     return cursor.fetchall()
19
20 # Function call
21 result = get_user("Amulya")
22 print(result)
23
24 conn.close()
```

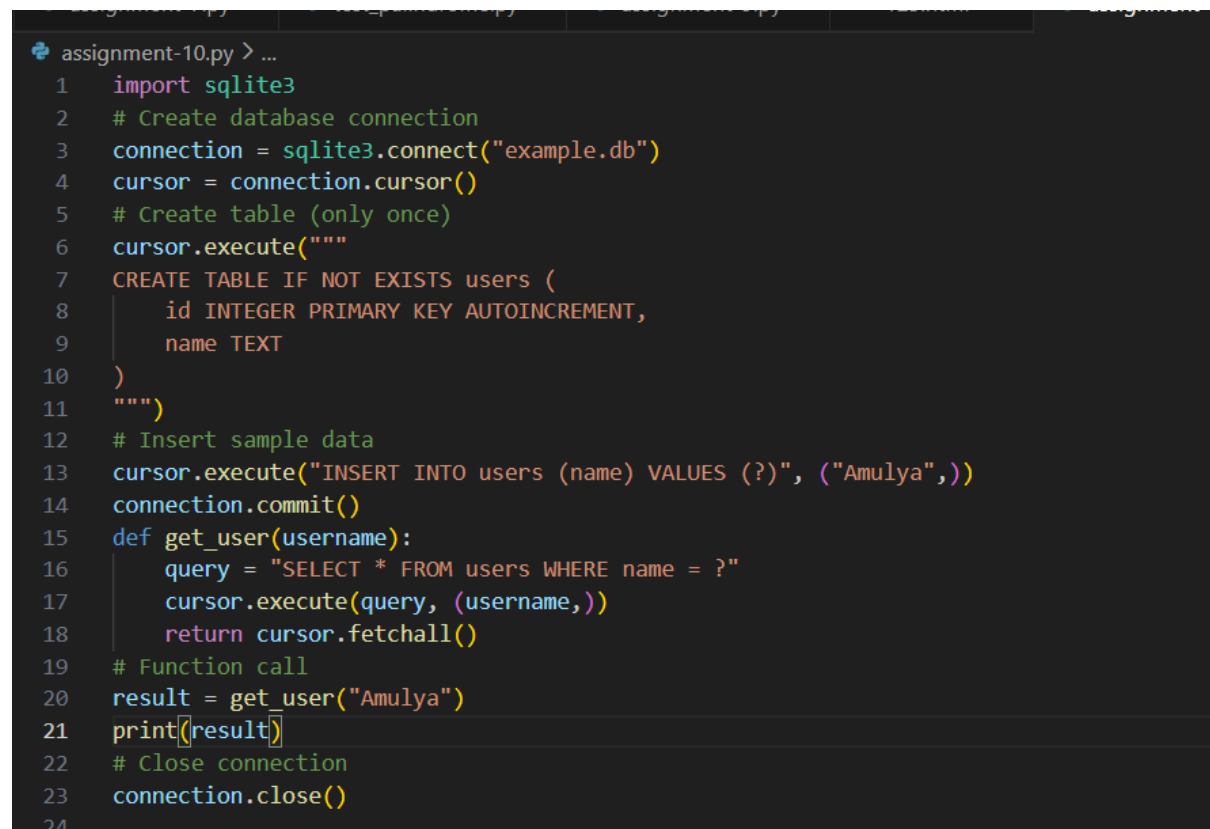
- Output:



```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-10.py"
['Amulya',,]
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

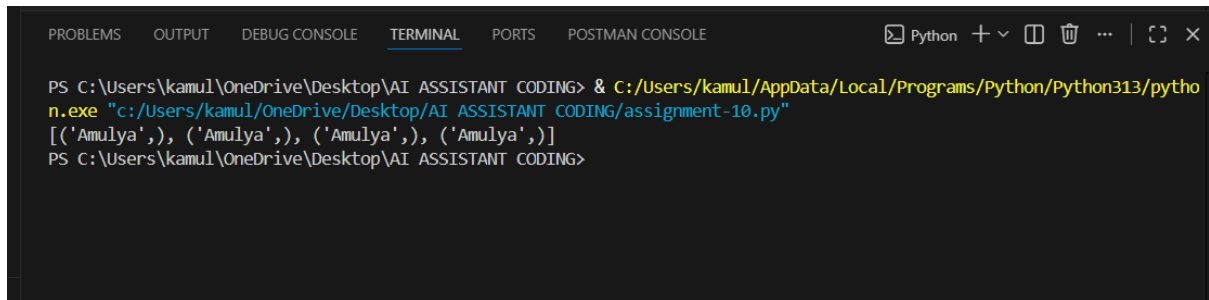
- Uses **string concatenation** for SQL queries
- Vulnerable to **SQL Injection**
- No input validation
- No exception

Secure Refactored Code:



```
assignment-10.py > ...
1  import sqlite3
2  # Create database connection
3  connection = sqlite3.connect("example.db")
4  cursor = connection.cursor()
5  # Create table (only once)
6  cursor.execute("""
7  CREATE TABLE IF NOT EXISTS users (
8      id INTEGER PRIMARY KEY AUTOINCREMENT,
9      name TEXT
10 )
11 """)
12 # Insert sample data
13 cursor.execute("INSERT INTO users (name) VALUES (?)", ("Amulya",))
14 connection.commit()
15 def get_user(username):
16     query = "SELECT * FROM users WHERE name = ?"
17     cursor.execute(query, (username,))
18     return cursor.fetchall()
19 # Function call
20 result = get_user("Amulya")
21 print(result)
22 # Close connection
23 connection.close()
24
```

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
Python + - [ ] [ ] ... | [ ] [ ] X

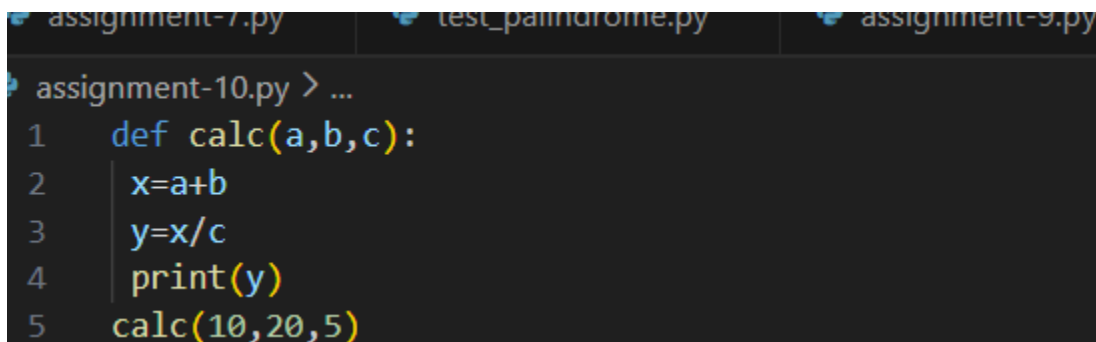
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-10.py"
[('Amulya',), ('Amulya',), ('Amulya',), ('Amulya',)]
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation:

- Added database connection
- Created table and sample data
- Used **parameterized query** (prevents SQL injection)
- Printed the function output
- Code now **executes successfully**

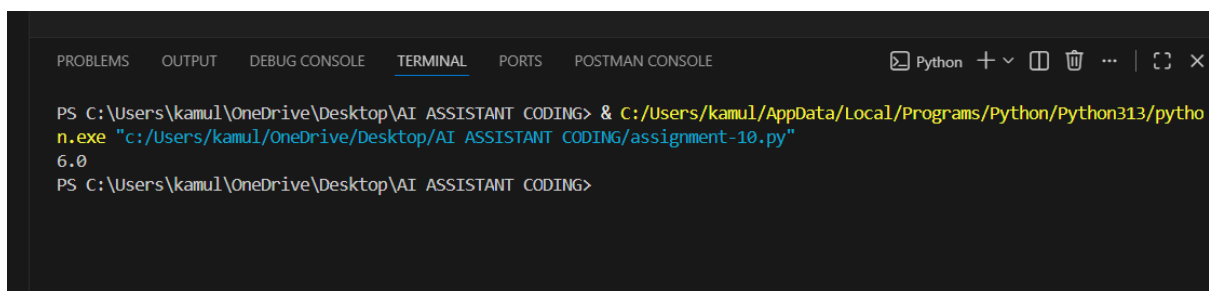
Task 5: AI-Based Automated Code Review Report

Poor code:



```
assignment-7.py test_palindrome.py assignment-9.py
assignment-10.py > ...
1 def calc(a,b,c):
2     x=a+b
3     y=x/c
4     print(y)
5 calc(10,20,5)
```

Output:



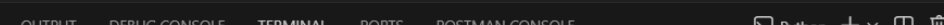
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
Python + - [ ] [ ] ... | [ ] [ ] X

PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-10.py"
6.0
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```


Improved code:

```
assignment-10.py > ...
1 def calculate_average(value1, value2, divisor):
2     """
3     Calculates the average of two numbers divided by a given divisor.
4     """
5     if divisor == 0:
6         raise ValueError("Divisor cannot be zero")
7
8     total = value1 + value2
9     result = total / divisor
10    print(result)
11    return result
12
13 # Example usage
14 calculate_average(10, 20, 5) # Output: 6.0
```

Output:



The screenshot shows a Windows command prompt window with the following text:

```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-10.py"
6.0
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

The command prompt shows the execution of a Python script named `assignment-10.py` using the Python 3.13 interpreter. The output of the script is `6.0`.

Explanation:

- Identified code smells and risk areas
- Suggested readability and safety improvements
- Demonstrated AI's role as a **code reviewer**, not just a generator